

Object-Oriented Analysis and UML-Based Design of an Intelligent Food Waste Reduction and Redistribution System (IFWRRS)

CSA11 – Object Oriented Analysis and Design

Table of Contents

1. Problem Analysis and Requirement Summary

1.1 Problem Overview

Large quantities of edible surplus food generated daily by restaurants, hotels, supermarkets, and food suppliers go to waste while charitable organizations, shelters, and community kitchens face food shortages. The **Intelligent Food Waste Reduction and Redistribution System (IFWRRS)** is proposed to bridge this gap by digitally connecting food providers, recipient organizations, delivery partners, and administrators, enabling timely, traceable, and efficient redistribution of surplus food before it expires.

1.2 Functional Requirements (derived from the scenario)

ID	Requirement
FR1	Registered food providers shall record surplus food with quantity, type, preparation time, expiry period, storage requirement, and pickup location.
FR2	The system shall identify suitable recipient organizations based on food requirement, location, quantity, and urgency (intelligent matching).
FR3	Recipient organizations shall view available donations, request items, confirm collection, and update distribution status.
FR4	Delivery partners shall receive pickup/delivery assignments and update transportation status.
FR5	Administrators shall monitor donations, organizations, deliveries, users, and generate reports.
FR6	The system shall generate notifications when surplus food is approaching expiry.
FR7	The system shall support user registration, authentication, and role-based access.

1.3 Non-Functional Requirements

- **Reliability:** Consistent state transitions for donation and delivery status.
- **Scalability:** Ability to add new providers, organizations, and regions without redesign.
- **Usability:** Simple interfaces for providers and NGOs with limited technical expertise.
- **Timeliness:** Near real-time matching and notification before food expiry.
- **Data consistency & transaction management:** Concurrent requests for the same donation must be handled safely.

1.4 Domain Summary

The domain centers on four stakeholder roles (Provider, Recipient Organization, Delivery Partner, Administrator) interacting around a central artifact — the **FoodDonation** — whose lifecycle (posted → matched → requested → confirmed → delivered → closed) drives most system behaviour.

2. Actor Identification

Actor	Type	Description
Food Provider	Primary, Human	Restaurant, hotel, supermarket, or supplier that posts surplus food donations.
Recipient Organization	Primary, Human	Charity, NGO, or shelter that requests and collects donations.
Delivery Partner	Primary, Human	Individual/agency that transports food from provider to recipient.
Administrator	Primary, Human	Oversees users, donations, deliveries, and system reports.
Matching Engine	Secondary, System	Automated component that matches donations to suitable recipients.
Notification System	Secondary, System	Automated component that issues expiry and status alerts.

3. Use Case Descriptions

UC-01: Post Surplus Food Donation

- **Actor:** Food Provider
- **Precondition:** Provider is registered and logged in.

- **Main Flow:** Provider enters food type, quantity, preparation time, expiry period, storage requirement, and pickup location → system validates data → system triggers Matching Engine (include) → donation status set to *Posted*.
- **Postcondition:** Donation is visible to matching recipient organizations.
- **Alternate Flow:** Invalid data → system prompts correction.

UC-02: Perform Intelligent Matching

- **Actor:** Matching Engine (system)
- **Trigger:** New donation posted, or periodic re-scan.
- **Main Flow:** Engine filters recipient organizations by food type, quantity capacity, location proximity, and urgency → ranks candidates → notifies top matches.

UC-03: Request Food Donation

- **Actor:** Recipient Organization
- **Precondition:** Donation is visible/matched.
- **Main Flow:** Organization views donation details → submits request → system notifies provider → includes UC-02 outcome as trigger context.

UC-04: Confirm Collection

- **Actor:** Recipient Organization, Food Provider
- **Main Flow:** Provider confirms the request → system creates a delivery assignment (include UC receive assignment) → recipient later confirms physical collection, closing the donation.

UC-05: Update Distribution Status

- **Actor:** Recipient Organization
- **Main Flow:** Organization marks donation as collected/distributed to end beneficiaries for reporting purposes.

UC-06: Receive Pickup/Delivery Assignment

- **Actor:** Delivery Partner
- **Main Flow:** System assigns nearest available partner → partner accepts → partner receives pickup and delivery location details.

UC-07: Update Transport Status

- **Actor:** Delivery Partner
- **Main Flow:** Partner updates status (Picked Up → In Transit → Delivered) as the delivery progresses; system notifies provider and recipient at each change.

UC-08: Monitor Donations, Organizations, Deliveries, Users

- **Actor:** Administrator
- **Main Flow:** Administrator views dashboards of all active/past donations, registered users, and ongoing deliveries; can suspend/approve accounts.

UC-09: Generate System Reports

- **Actor:** Administrator
- **Main Flow:** Administrator requests a report (e.g., waste reduced, donations per region) → system compiles and exports data.

UC-10: Send Expiry Notification

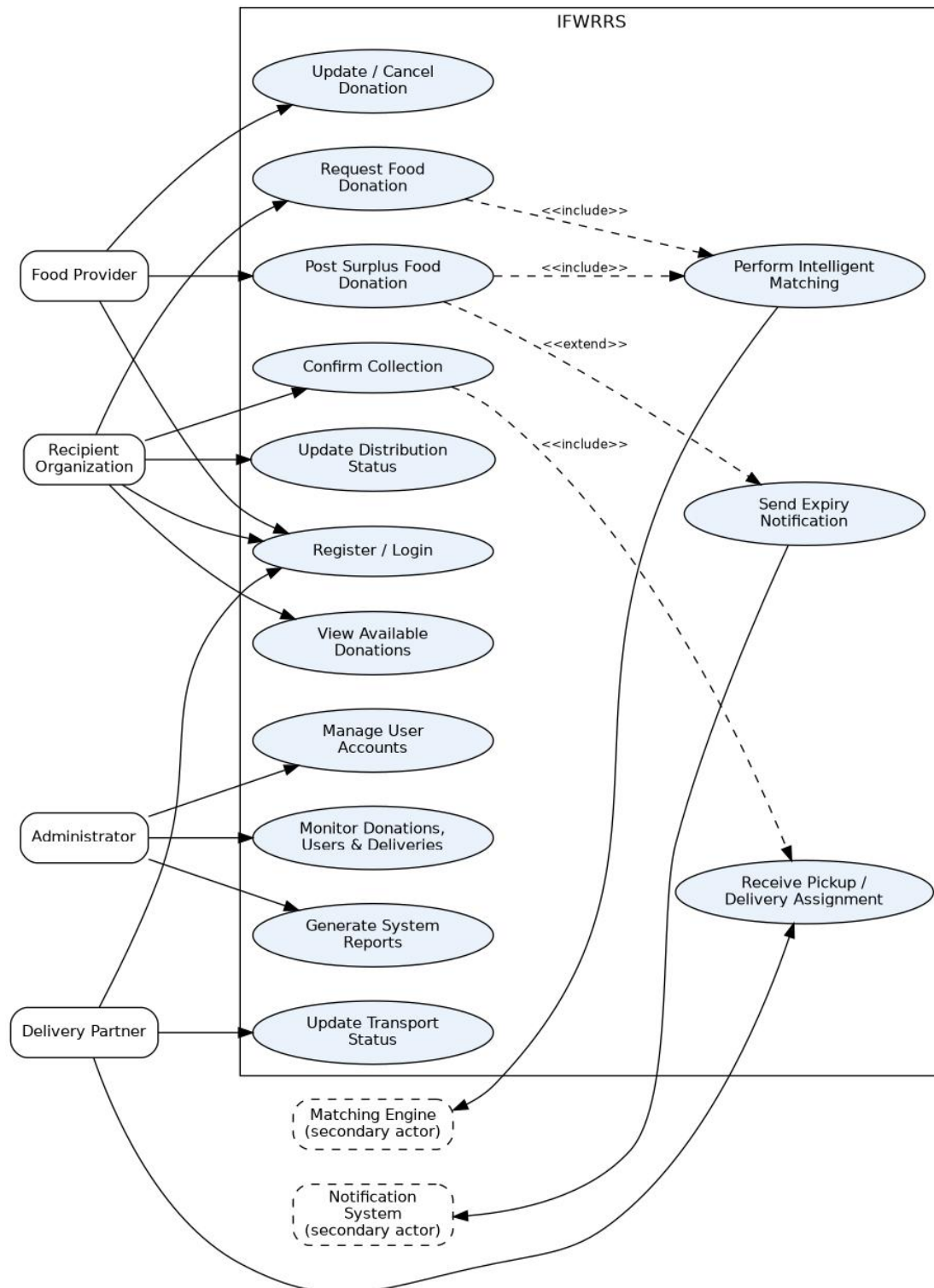
- **Actor:** Notification System (system), extends UC-01/UC-02
- **Main Flow:** When a posted donation nears its expiry threshold without being matched/collected, the system automatically alerts the provider and nearby recipient organizations.

UC-11: Register / Login

- **Actor:** All human actors
 - **Main Flow:** New users register with role-specific details; returning users authenticate via credentials.
-

4. Use Case Diagram

Use Case Diagram - IFWRRS



Use Case Diagram

The diagram shows four primary human actors interacting with the system boundary and two secondary system actors (Matching Engine, Notification System) that automate matching and alerting. <<include>> relationships capture mandatory sub-behaviour (e.g., posting a donation always includes matching); <<extend>> captures optional/conditional behaviour (expiry notification only fires under a condition).

5. Object and Class Identification

5.1 Candidate Objects (from noun analysis of the scenario)

Food provider, restaurant/hotel/supermarket/supplier, recipient organization, delivery partner, administrator, surplus food, quantity, food type, preparation time, expiry period, storage requirement, pickup location, recipient requirement, urgency, availability, donation request, collection, distribution status, delivery assignment, transportation status, notification, report.

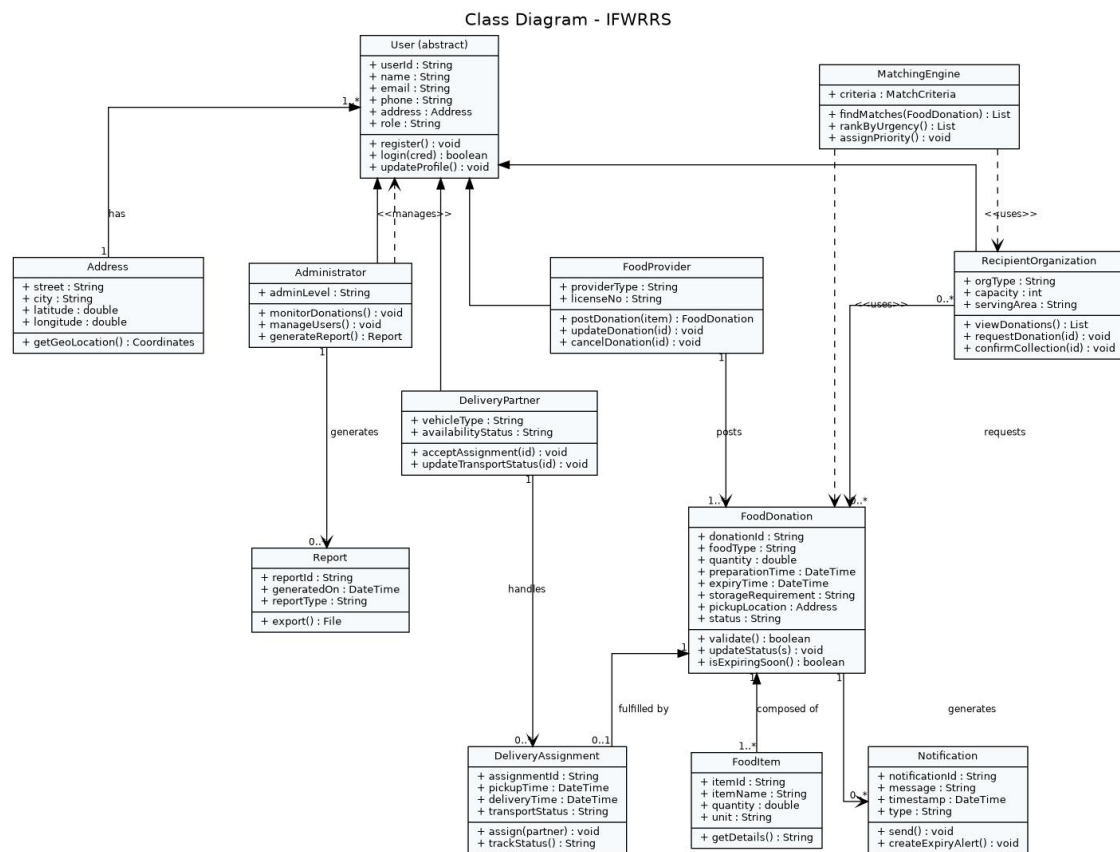
5.2 Classification of Objects into Classes

Applying object classification (behaviour + attribute clustering, and eliminating redundant/irrelevant candidates such as “quantity” which becomes an attribute, not a class), the following classes were identified:

Class	Category	Justification
User (abstract)	Entity	Common identity/auth attributes shared by all roles — supports inheritance and polymorphism.
FoodProvider, RecipientOrganization, DeliveryPartner, Administrator	Entity	Role-specialized behaviour that differs from the base User.
FoodDonation	Entity	Central business object; owns the lifecycle state machine.
FoodItem	Entity	Line-item detail when a donation bundles multiple food items.
Address	Entity/Value	Reusable location value object for pickup/delivery.
MatchingEngine	Control	Encapsulates matching algorithm — a control class,

Class	Category	Justification
Notification	Entity/Boundary	not persisted as a normal entity. Represents a message dispatched to a user.
DeliveryAssignment	Entity	Associates a donation with a delivery partner and tracks transport status.
Report	Entity	Administrator-facing aggregated output.

6. Class Diagram



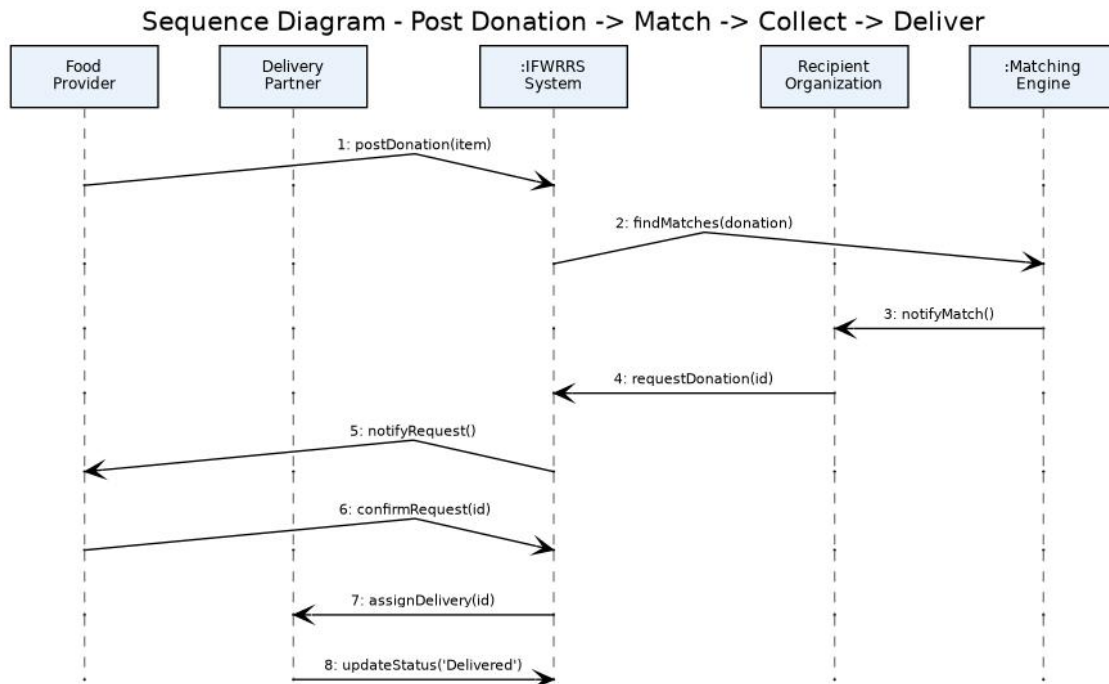
Class Diagram

Key relationships:

- **Inheritance (generalization):** FoodProvider, RecipientOrganization, DeliveryPartner, Administrator all generalize from abstract User.
- **Composition:** FoodDonation *owns* one or more FoodItems — items cannot exist without the donation (filled diamond).
- **Aggregation:** User *has* one or more Address — an address can conceptually be reused/exist independent of the

user record (hollow diamond). - **Association:** FoodProvider *posts* FoodDonation; RecipientOrganization *requests* FoodDonation; DeliveryPartner *handles* DeliveryAssignment. - **Dependency (<<uses>>):** MatchingEngine depends on FoodDonation and RecipientOrganization data without owning them.

7. Sequence / Interaction Diagram

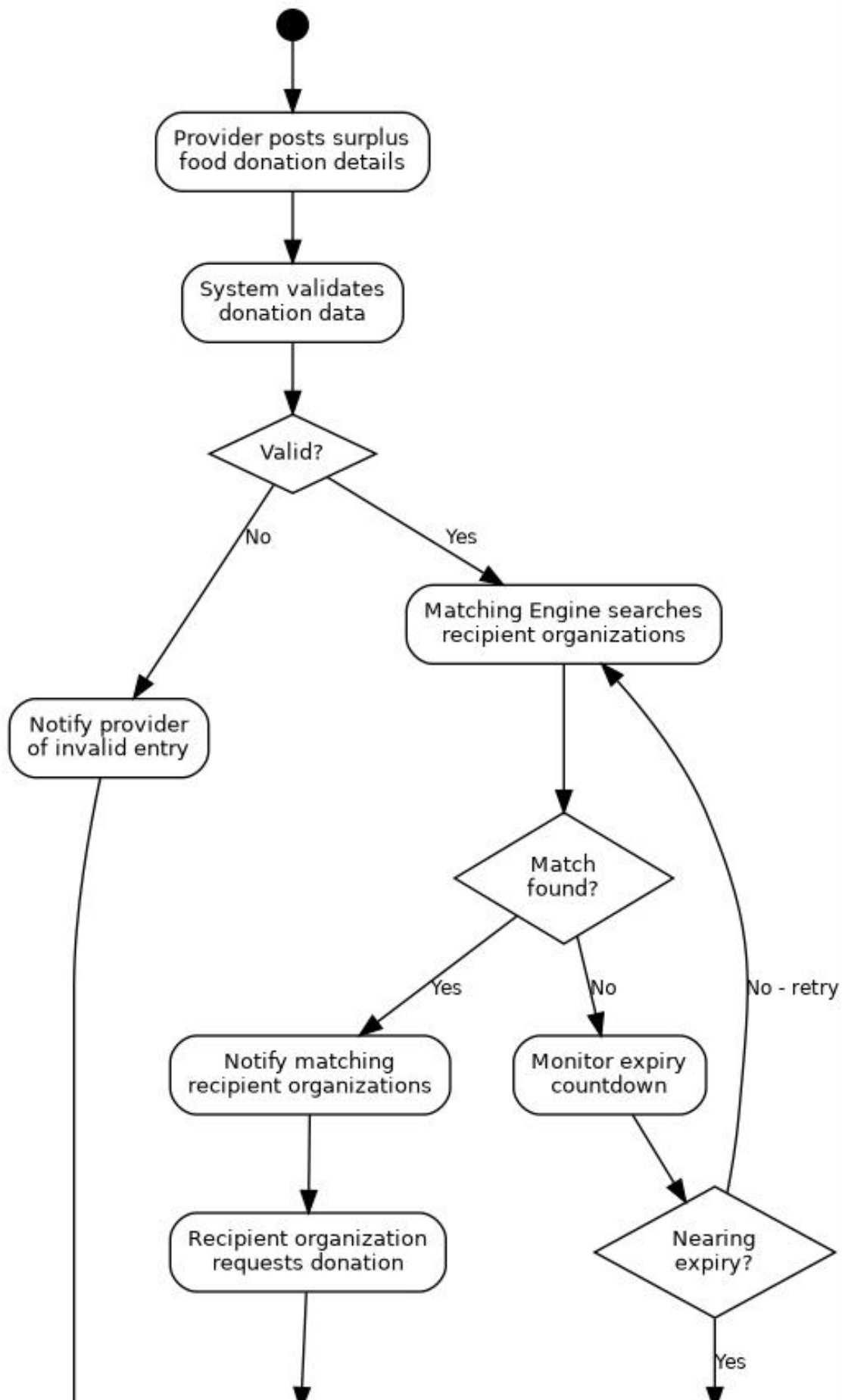


Sequence Diagram

This interaction diagram traces the primary success scenario for **UC-01 → UC-02 → UC-03 → UC-04 → UC-06 → UC-07**: a provider posts a donation, the system validates and triggers matching, the recipient organization requests it, the provider confirms, and the delivery partner is assigned and updates transport status through to delivery.

8. Activity Diagram

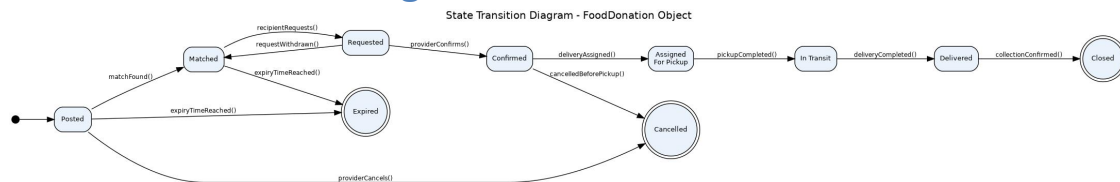
Activity Diagram - Food Donation Lifecycle



Activity Diagram

The activity diagram models the complete **FoodDonation lifecycle**, including the decision points for data validity, match availability, and expiry monitoring — showing both the happy path (posted → delivered → closed) and the expiry path (posted → unmatched → expired).

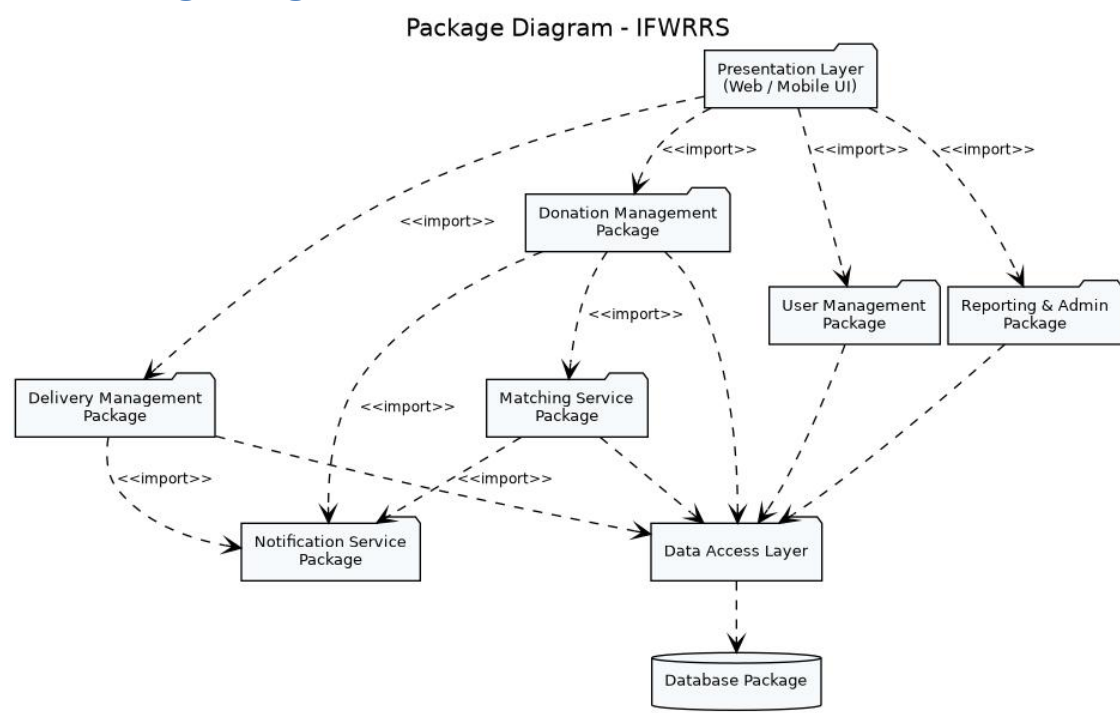
9. State Transition Diagram



State Transition Diagram

The FoodDonation object exhibits nine distinct states. Guard conditions (e.g., expiryTimeReached()) ensure a donation cannot silently remain active past its safe consumption window, directly satisfying the constraint on food-expiry handling.

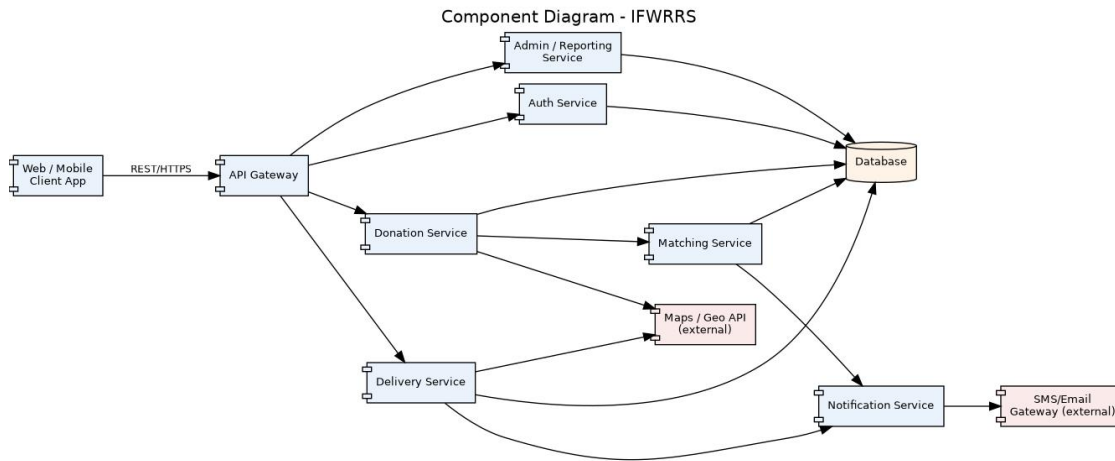
10. Package Diagram



Package Diagram

The system is organized into a **layered architecture**: a Presentation layer, four cohesive business-logic packages (User Management, Donation Management, Matching Service, Delivery Management, Notification Service, Reporting & Admin), and a Data Access layer over a single Database package. Dependencies flow strictly downward, minimizing coupling between the UI and persistence concerns.

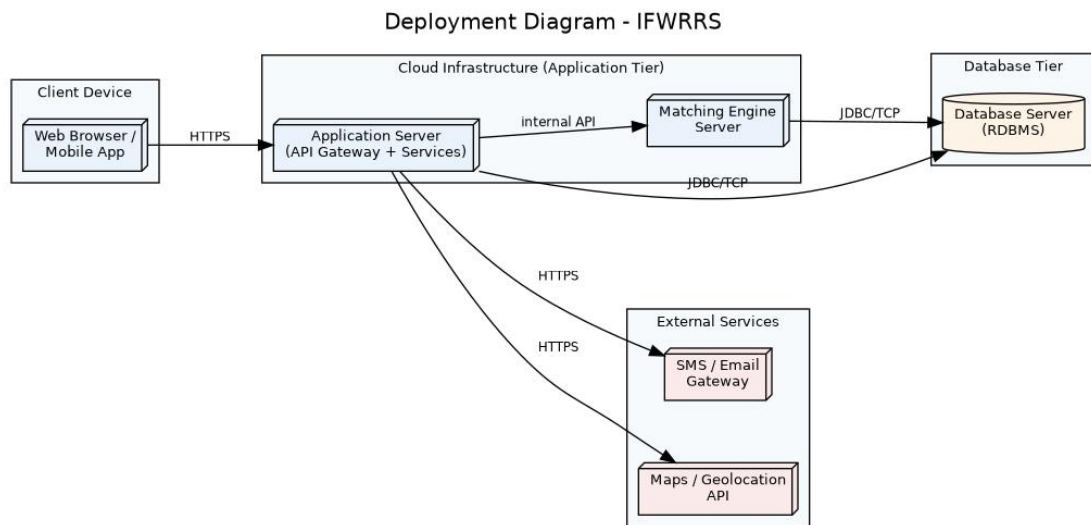
11. Component Diagram



Component Diagram

Each business package is realized as an independently deployable **service component** behind an API Gateway, communicating over REST/HTTPS. External dependencies (SMS/Email Gateway, Maps/Geo API) are isolated behind the Notification and Donation/Delivery services respectively, so they can be replaced without affecting core logic.

12. Deployment Diagram



Deployment Diagram

Client devices communicate with a cloud-hosted application-server tier (API Gateway + services) over HTTPS. The Matching Engine is deployed as a separate node to allow independent scaling of computationally heavier matching jobs. A dedicated database server and external SMS/Maps services complete the physical topology.

13. Object Responsibility Analysis (CRC Cards)

Class	Responsibilities	Collaborators
FoodProvider	Post/update/cancel donations; maintain profile	FoodDonation, Notification
RecipientOrganization	Browse donations; request; confirm collection	FoodDonation, MatchingEngine, Notification
DeliveryPartner	Accept assignment; report transport status	DeliveryAssignment, Notification
Administrator	Monitor entities; manage accounts; generate reports	User, FoodDonation, Report
FoodDonation	Hold donation data; enforce valid status transitions; detect nearing expiry	FoodItem, Notification, DeliveryAssignment
MatchingEngine	Filter and rank recipient organizations for a donation	FoodDonation, RecipientOrganization
DeliveryAssignment	Link a donation to a delivery	FoodDonation,

Class	Responsibilities	Collaborators
	partner; track transport progress	DeliveryPartner
Notification	Compose and dispatch a message to the relevant user(s)	User
Report	Aggregate and export operational data	FoodDonation, DeliveryAssignment

Responsibility assignment follows the **GRASP** principles: *Information Expert* (FoodDonation owns its own status-transition logic because it holds the state), *Creator* (FoodProvider creates FoodDonation instances since it supplies the data), *Controller* (MatchingEngine acts as a use-case controller rather than embedding matching logic inside entity classes), and *Low Coupling / High Cohesion* (Notification is a single-purpose class reused by every other component instead of each class emailing/SMS-ing directly).

14. Design Axioms, Principles and Design Pattern Justification

14.1 Design Axioms (Suh's Axiomatic Design, applied conceptually)

- **Independence Axiom:** Each package (User Management, Donation Management, Matching, Notification, Delivery, Reporting) maps to an independent functional requirement, so a change to matching logic does not ripple into delivery tracking.
- **Information Axiom:** Among designs satisfying the independence axiom, the simplest one (fewest classes/relationships) was preferred — e.g., a single Notification class serves all alert types instead of one subclass per alert type.

14.2 Object-Oriented Design Principles Applied

- **High Cohesion:** FoodDonation only manages donation data and its own lifecycle; delivery tracking is delegated to DeliveryAssignment.
- **Low Coupling:** UI/Presentation never talks to the database directly — always via service packages, per the layered Package Diagram.
- **Encapsulation:** State transition rules are private to FoodDonation.update_status(); no external class mutates status directly.
- **Abstraction:** User is an abstract class exposing a common contract (register(), role()), hiding role-specific implementation details.
- **Open/Closed Principle:** New provider or organization sub-types can be added by extending User without modifying existing matching or notification logic.

14.3 Design Patterns Justified

Pattern	Where Applied	Justification
State Pattern	FoodDonation status lifecycle	Encapsulates legal transitions (Posted→Matched→...→Closed) and prevents invalid states, directly satisfying the “data consistency” constraint.
Strategy Pattern	MatchingEngine	The matching algorithm (proximity, urgency, capacity) can be swapped without changing the classes that call it.
Observer Pattern	Notification dispatch	Providers, recipients, and delivery partners “subscribe” to status changes on a donation and are notified automatically.
Factory Method	User registration	Creates the correct User subclass (FoodProvider, RecipientOrganization, etc.) based on the selected role at registration.
Singleton (candidate)	Database connection manager	Ensures a single shared connection pool across all service components, avoiding resource duplication.
MVC (architectural)	Overall layering	Presentation (View), Service packages (Controller), Entity classes (Model) — supports the Package/Component diagrams.

15. Assumptions and Constraints

15.1 Assumptions

1. All actors access the system through a single web/mobile client; no offline mode is required.
2. A donation is assumed to be collected by exactly one recipient organization (no partial-quantity splitting in this iteration).
3. Delivery partners are either gig-workers or NGO-affiliated volunteers, already onboarded with basic verification.
4. Expiry threshold for triggering alerts is configurable but defaults to 30 minutes before expiryTime.
5. Location data (latitude/longitude) is available for all providers, organizations, and pickup points.

15.2 Constraints

1. All classes, attributes, and operations must trace back to at least one use case (Section 16).

2. UML diagrams must remain mutually consistent — e.g., every message in the Sequence Diagram corresponds to an operation in the Class Diagram.
3. The design must account for food expiry, notification timing, matching criteria, delivery status, user roles, data consistency, and transaction management, per the assignment brief.
4. The system must distinguish stakeholder responsibilities clearly (no shared/God classes).

16. Requirement-to-Use Case/Class Traceability Matrix

Requirement	Use Case(s)	Primary Class(es)
FR1 – Record surplus food	UC-01	FoodProvider, FoodDonation, FoodItem
FR2 – Intelligent matching	UC-02	MatchingEngine, FoodDonation, RecipientOrganization
FR3 – View/request/confirm	UC-03, UC-04, UC-05	RecipientOrganization, FoodDonation
FR4 – Pickup/delivery & status	UC-06, UC-07	DeliveryPartner, DeliveryAssignment
FR5 – Admin monitoring & reports	UC-08, UC-09	Administrator, Report, User
FR6 – Expiry notifications	UC-10	Notification, FoodDonation
FR7 – Registration/auth	UC-11	User (and subclasses)

17. Implementation / Prototype and Results

A console-based Python prototype was implemented to validate the core object model — the User inheritance hierarchy, the FoodDonation state machine, and the MatchingEngine strategy — before full UI/database implementation. The prototype enforces valid state transitions exactly as modelled in the State Transition Diagram (Section 9), raising an error on any illegal transition attempt.

Sample execution output:

```
=====
IFWRRS PROTOTYPE DEMONSTRATION
=====
FoodProvider 'Green Leaf Hotel' registered with id USR0001
RecipientOrganization 'Hope Shelter' registered with id USR0002
DeliveryPartner 'Ravi Kumar' registered with id USR0003
Administrator 'System Admin' registered with id USR0004
```


Green Leaf Hotel posted donation DNT0005: 40 kg of Cooked Rice & Curry (expires in 120 min)

[Notification -> INFO] Donation DNT0005 status changed to 'Matched'
MatchingEngine found 1 match(es) for DNT0005: ['Hope Shelter']

[Notification -> INFO] Donation DNT0005 status changed to 'Requested'
Hope Shelter requested donation DNT0005

[Notification -> INFO] Donation DNT0005 status changed to 'Confirmed'

[Notification -> INFO] Donation DNT0005 status changed to 'AssignedForPickup'

Ravi Kumar updated assignment DLV0009 -> Picked Up

[Notification -> INFO] Donation DNT0005 status changed to 'InTransit'

Ravi Kumar updated assignment DLV0009 -> Delivered

[Notification -> INFO] Donation DNT0005 status changed to 'Delivered'

[Notification -> INFO] Donation DNT0005 status changed to 'Closed'

Admin System Admin monitoring 1 donation(s):

- DNT0005: Closed

Final status of DNT0005: Closed

Result: The donation object transitioned correctly through all nine lifecycle states with no invalid transitions, and every status change automatically generated a Notification, confirming that the Observer-style notification design and the State-pattern-based lifecycle work as modelled.

18. Reflection on Design Decisions and Learning Outcomes

Modelling IFWRRS reinforced how object identification (Section 5) is rarely a one-pass exercise — several early candidate classes (e.g., “Urgency”, “Availability”) were correctly demoted to attributes or matching criteria rather than standalone classes, which sharpened the distinction between *entity*, *attribute*, and *control* responsibilities. Choosing the **State pattern** for FoodDonation was the most consequential design decision: centralising transition rules directly satisfies the assignment’s data-consistency constraint and made the Activity, Sequence, and State diagrams naturally consistent with one another, since every diagram references the same fixed vocabulary of statuses.

The exercise also clarified the practical value of **GRASP** and **SOLID** principles beyond theory — e.g., delegating matching logic to a dedicated MatchingEngine (Controller/Strategy) rather than embedding it inside FoodDonation kept that class cohesive and made the algorithm independently testable and replaceable.

Relevance to selected SDGs: IFWRRS directly supports **SDG 2 (Zero Hunger)** by redirecting edible surplus to those in need, **SDG 12 (Responsible Consumption**

and Production) by reducing avoidable food waste at the source, **SDG 11 (Sustainable Cities and Communities)** by strengthening local food-redistribution networks, and **SDG 13 (Climate Action)** indirectly, since diverting food from disposal reduces the methane emissions associated with landfill decomposition.

19. GitHub Upload Checklist

- ☐ Create a repository named IFWRRS-OOAD-Assignment.
- ☐ Add README.md summarizing the problem statement and this document's structure.
- ☐ Upload all .dot/PlantUML diagram sources under /diagrams/src and rendered .png files under /diagrams/img.
- ☐ Upload prototype.py under /prototype.
- ☐ Upload this report (.docx/.pdf) under /docs.
- ☐ Commit with a descriptive message, e.g. "Add OOAD analysis, UML diagrams, and prototype for IFWRRS".
- ☐ Share the repository link in the final submission.